

CST2550 Reset Coursework Report

Project: Local Supermarket Management System for Small Shops

Student: Thriambakeshwar Manjunath Student ID: M01046020

1. Introduction

This project is a console supermarket management system based on C# .Net. The aim is to replace a paper system in a small shop by a simple digital system for products, suppliers, stock and sales. I kept the menu simple as the user will most likely be a shop worker who needs to add stock, check products and record a sale quickly as opposed to using a complicated interface.

The program can create and update products, manage suppliers, update and restock inventory, record sales, send low-stock alerts, and generate basic reports. When LocalDB is available, persistence is done using SQL Server with Entity Framework Core. If the database is not on the marking machine then the program opens with starter data so that the main features are easy to show.

As the system is targeted to a small supermarket I concentrated on the daily shop tasks instead of the more advanced retail features. This is not intended to be a full EPOS system. It is a small management tool, which demonstrates how data structures, search algorithms and a relational database can work together in one coursework project.

2. Data structure design

The main in-memory structure is a set of singly linked lists. I used ProductLinkedList for products, SupplierLinkedList for suppliers and SaleLinkedList for sales. Each list has a node class with a data field and a

Next pointer. This makes the structure visible in the code and avoids relying on List<T> as the main product/supplier/sales structure. List<T> only appears in EF navigation properties because Entity Framework uses collection navigation fields for relationships.

A linked list is a good choice here, because a new product or supplier can be inserted without having to resize an array. The weakness is searching, as a linked list has to go through records one by one. I added a small array of barcode indexes created from the linked list and sorted using insertion sort to speed up barcode searching. This is a practical compromise: the main structure remains the linked list but binary search can be used to display the barcode lookup.

Products have the necessary fields for supermarkets. These include unique ID, barcode, title, category, supplier, price, quantity in stock, stock status and expiry/restock date. Suppliers have contact information, and sales store the date and total amount. Stock is also stored in the database as a separate table so the SQL design matches the brief.

3. Algorithms and time complexity

The main search methods are linear search and binary search. Linear search is used for product name searches

because the linked list is not ordered by name. Binary search is used for barcode searches after building a sorted barcode index. Insertion sort is used because it is simple to show and fits the coursework focus on manually implemented algorithms.

Operation	Method used	Time complexity
Add product	Validate, check ID/barcode, add to linked list	O(n)
Search by name	Linear search through ProductLinkedList	O(n)
Prepare barcode index	Insertion sort on product array	O(n ²)
Search by barcode	Binary search on sorted barcode index	O(log n) after index refresh
Update stock	Find product then update quantity/status	O(n)
Record sale	Find product, check stock, deduct quantity	O(n)
Low-stock report	Scan all products and copy matches	O(n)

Binary search pseudocode

left = 0, right = length - 1. While left <= right, calculate middle. If the middle barcode matches, return the product. If the middle barcode is smaller, move left to middle + 1. Otherwise move right to middle - 1. The sorted index is only rebuilt after product data changes, so repeated barcode searches do not sort every time.

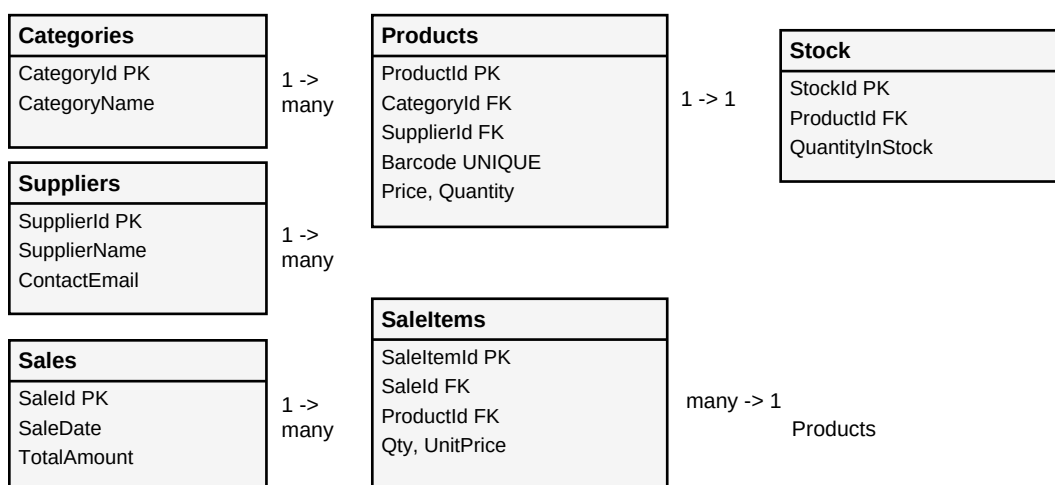
4. Database design and persistence

The database contains six required tables: Products, Categories, Suppliers, Stock, Sales and SaleItems. Products are linked to Categories and Suppliers using foreign keys. Stock has a one-to-one link with Product, and SaleItems link each sale back to both Sales and Products. The SQL scripts in the Database folder can create and seed the same structure manually.

Entity Framework Core is configured in SupermarketDbContext. Decimal fields use decimal(10,2), barcode/category/supplier names are indexed where needed, and delete behaviour is configured so a product cannot be removed in a way that breaks sales history.

5. Database relationship diagram

Figure 1 shows the simple relational design used by the system. Products are central because categories, suppliers, stock and sale item records all connect to product records.



6. Validation, testing and user interface

The console menu has options for products, suppliers, stock, sales and reports. Validation checks include required fields, positive product IDs, positive prices, non-negative stock, valid supplier contact details and duplicate ID/barcode checks. A sale is rejected if there is not enough stock, and stock status is refreshed after sale or restock changes.

Test area	Examples covered
Product tests	valid product, duplicate barcode, negative stock
Search tests	linear search by name and binary search by barcode
Stock tests	stock update and restock
Sales tests	successful sale and insufficient stock
Report tests	low-stock report and category report

```
=====
LOCAL SUPERMARKET STOCK AND SALES SYSTEM
Student: Thriambakeshwar Manjunath | ID: M01046020
Storage: Sample list only
=====
1. Add product
2. Edit product
3. Remove product
4. View product list
5. Add supplier
6. Edit supplier
7. Remove supplier
8. View suppliers
9. Find product by name
10. Find product by barcode
11. Set stock quantity
12. Add restock quantity
13. Record sale
14. Reports
0. Exit
=====
Choose an option: 1

ADD PRODUCT
Product ID: 101
Barcode: 8901234567890
Product title: Dairy Milk Chocolate
Category: snacks
Supplier name: Thriam
Price: £1.50
Quantity in stock: 25
Expiry/restock date (yyyy-mm-dd): 2026-12-31
Dairy Milk Chocolate has been added to the product list.

Press Enter to continue...
|
```

Product added successfully with barcode, price and stock quantity.

```
=====
LOCAL SUPERMARKET STOCK AND SALES SYSTEM
Student: Thriambakeshwar Manjunath | ID: M01046020
Storage: Sample list only
=====
1. Add product
2. Edit product
3. Remove product
4. View product list
5. Add supplier
6. Edit supplier
7. Remove supplier
8. View suppliers
9. Find product by name
10. Find product by barcode
11. Set stock quantity
12. Add restock quantity
13. Record sale
14. Reports
0. Exit
=====
Choose an option: 13

Product ID sold: 101
Quantity sold: 13
Sale saved for 13 x Dairy Milk Chocolate. Total: £19.50
Receipt total: £19.50

Press Enter to continue...
|
```

Sale recorded successfully for Dairy Milk Chocolate with receipt total calculated.

7. Limitations and reflection

The drawback is that currently a sale can only be made for one product line at a time and not for a full basket containing multiple items. This was a conscious simplification to keep the sales, stock deduction and reporting clear in a six-minute demo. Another limitation is the barcode index. It works for demonstrating binary search but in a larger commercial system I would use a database index or a dictionary style lookup.

Originally I kept the project very focused on the linked lists, but after checking the brief again, I connected the program to SQL Server too. That was the main improvement, because a real shop would lose data if everything stayed only in memory. I also added the in-memory fallback because I did not want the whole demonstration to fail just because LocalDB was not installed. The final version is still simple, but it supports the core operations needed by a small supermarket: products, suppliers, stock, sales, searches and reports. Future improvements would include receipt printing, barcode scanner support, customer accounts, multi-item baskets and more detailed sales analytics.

References

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) Introduction to Algorithms. 4th edn. Cambridge, MA: The MIT Press. Available at: <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/> (Accessed: 24 June 2026).

Microsoft (2024) Overview of Entity Framework Core. Available at: <https://learn.microsoft.com/en-us/ef/core/> (Accessed: 24 June 2026).

Microsoft (2026) C# documentation. Available at: <https://learn.microsoft.com/en-us/dotnet/csharp/> [Accessed 24 June 2026]

.

Microsoft (2026) .NET documentation. Available at: <https://learn.microsoft.com/en-us/dotnet/> [Accessed 24 June 2026].

xUnit.net (2026) Getting Started with xUnit.net v3. Available at: <https://xunit.net/docs/getting-started/v3/getting-started> (Accessed: 24 June 2026).